

 [davidarchanjo](#) / [spring-certified-developer-study-guide](#) Public

My study guide for the Spring Certified Developer certification

☆ 60 stars 🍴 30 forks

☆ Star

🔔 Notifications

<> Code

🔍 Issues

🔗 Pull requests

💬 Discussions

🎬 Actions

📁 Projects

🛡 Security

📈 Insights

🔑 main ▾

Go to file



davidarchanjo Merge pull request #2 from CodePawfect/main ...

4 days ago 🔁 285

[View code](#)

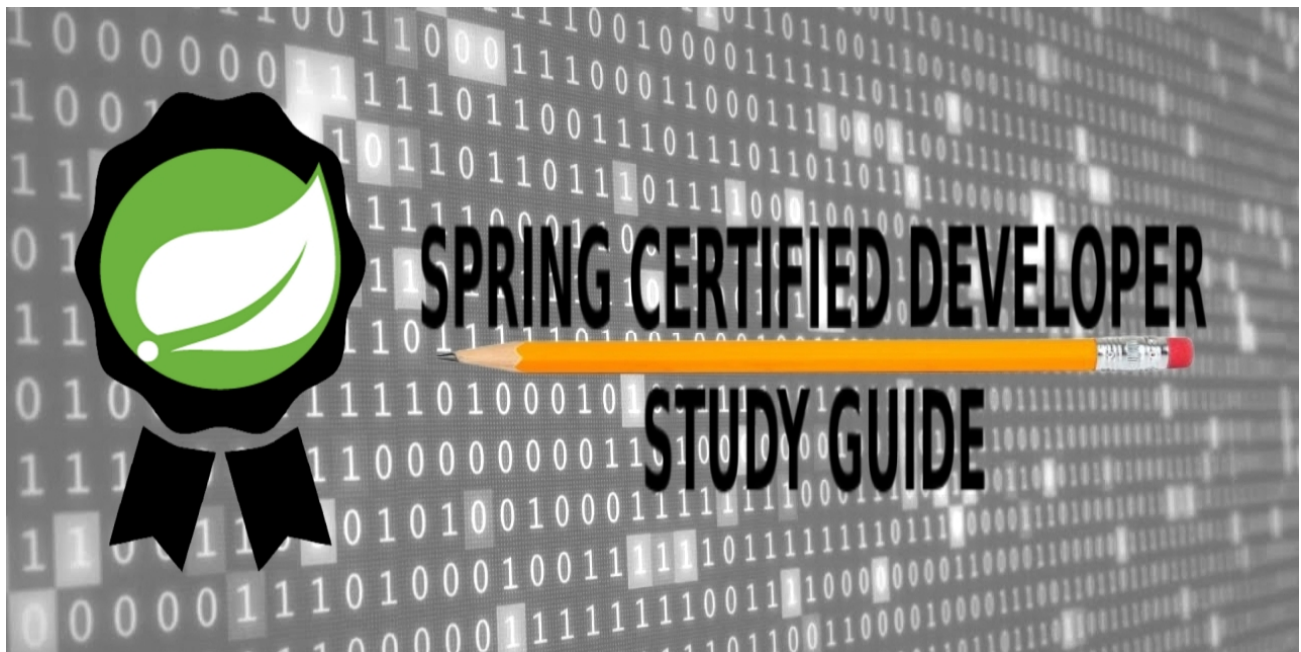


TABLE OF CONTENTS

1. [INTRODUCTION](#)
2. [EXAM OVERVIEW](#)
3. [REST CONCEPTS](#)

4. [SPRING CORE CONCEPTS](#)
5. [SPRING BOOT BASICS AND AUTO-CONFIGURATION](#)
6. [ASPECT-ORIENTED PROGRAMMING](#)
7. [DATA MANAGEMENT: JDBC, TRANSACTIONS](#)
8. [SPRING DATA JPA](#)
9. [SPRING MVC](#)
10. [SPRING SECURITY](#)
11. [SPRING BOOT ACTUATOR](#)
12. [SPRING BOOT TESTING](#)
13. [BUILD & TEST](#)

1. INTRODUCTION

This guide walks through some technical notes and references about the Spring Framework, with focus on the Spring Boot Framework, serving as a *supporting material* for whoever is preparing to take the [Spring Certified Professional](#) exam. I came up with this guide in order to consolidate my knowledge throughout my studies, hence all the content presented here are by no means definitive and exhaustive so that someone only based on what I present here can take the exam fully prepared. Ultimately this guide contains many sources for relevant technical documentations, tutorials and notes, and also offers a structured and concise study planning to help on your preparation.

2. EXAM OVERVIEW

The **Spring Certified Professional** certification offered by VMWare is an exam scaled on a range from 100-500, with the determined raw cut score scaled to a value of 300. The exam may contain unscored questions in addition to the scored questions, this is a standard testing practice used by VMWare. We can't know which questions are unscored and the final result will reflect the scored questions only. This certification exam is designed to test and validate the overall understanding and familiarity with core aspects of Spring and Spring Boot frameworks as follows:

- Container, Dependency, and IoC
- Aspect-Oriented Programming (AOC)
- Data Management: JDBC, Transactions
- Spring Data JPA
- Spring MVC Basics

- Spring MVC REST
- Security
- Testing
- Spring Boot Basics
- Spring Boot Auto Configuration
- Spring Boot Actuator
- Spring Boot Testing

EXAM INFO

Duration: 90 Minutes

Number of Questions: 50 Questions

Passing Score: 300

Format: Multiple Choice, Multiple Choice Multiple Selection, Drag and Drop, Matching

3. REST CONCEPTS

REFERENCES

- <https://restfulapi.net>
- <https://restapitutorial.com>

OVERVIEW

REST stands for **RE**presentational **S**tate **T**ransfer and it is a software architectural style introduced by [Roy Fielding](#) on 2000 as a design guide to the WEB on its early days.

Any web API that conforms to [The REST Principles](#) is classified as **RESTful**.

The data and its functionalities provided by a RESTful API are referred *resources* and are accessed through **Uniform Resource Identifiers**.

HTTP METHODS

REST API resources are manipulated by HTTP methods and the most-commonly-used HTTP methods are POST, GET, PUT and DELETE. These methods correspond, respectively, to **Create**, **Read**, **Update**, and **Delete** operations and those actions are commonly referred by the **CRUD** acronym.

The following summarises the use of those HTTP methods:

Method	Action
POST	Create a resource
GET	Retrieve information about a specific resource or a collection
PUT	Update a resource
DELETE	Delete a resource or related component

HTTP STATUS

In REST APIs, HTTP status codes are used to inform the consumer about the request's result. The status codes are divided into five categories:

Category	Description
1xx Status	Indicates that the server received the request and is continuing the process
2xx Status	Indicates that the client's request was accepted successfully
3xx Status	Indicates that the client must take some additional action in order to complete the request
4xx Status	Indicates that the client's request contains errors or bad syntax
5xx Status	Indicates that the client's request was unable to be performed due to a server-side error

API CONVENTIONS

The key principles of REST involves separating the API into *logical resources* and manipulate them through HTTP requests where every HTTP method (GET, POST, PUT, DELETE etc) performs a specific operation.

The functionalities provided by an API are related to *resources* and by convention the resources are represented on URIs as **nouns in the plural**. All operation made available via URI should be mapped to a HTTP method that corresponds to the action is intended to be carried out.

Follows some common examples of mapping correspondence between the HTTP method, operation type and URI format:

Method	URL	Operation
GET	/todos	Retrieves a list of all todos
GET	/todos/{id}	Retrieves todo referenced by <code>id</code>
POST	/todos	Creates a new todo
PUT	/todos/{id}	Fully updates todo referenced by <code>id</code>
PATCH	/todos/{id}	Partially updates todo referenced by <code>id</code>
DELETE	/todos/{id}	Deletes todo referenced by <code>id</code>

4. SPRING CORE CONCEPTS

SAMPLE PROJECTS

- [spring-bean](#)
- [spring-bean-factory-post-processor](#)
- [spring-bean-lifecycle-callback-order](#)
- [spring-bean-post-processor](#)
- [spring-spel](#)
- [spring-startup](#)

REFERENCES

- <https://www.baeldung.com/inversion-control-and-dependency-injection-in-spring>
- <https://knpcode.com/spring/spring-ioc-container-types-applicationcontext-and-beanfactory>
- <https://www.journaldev.com/2410/spring-dependency-injection>
- <https://www.baeldung.com/spring-bean>
- <https://www.baeldung.com/spring-bean-scopes>
- <https://dzone.com/articles/spring-bean-lifecycle>
- <https://www.geeksforgeeks.org/bean-life-cycle-in-java-spring>
- <https://knpcode.com/spring/spring-bean-lifecycle-callback-methods>
- <https://www.baeldung.com/spring-core-annotations>
- <https://www.baeldung.com/spring-expression-language>

- <https://asyncstream.com/tutorials/spring-initmethod-or-initializingbean/>
- <https://programmer.help/blogs/after-properties-set-and-init-method-postconstruct-of-spring-initializing-bean.html>
- <https://www.dineshonjava.com/writing-beanpostprocessor-in-spring/>
- <https://www.dev2qa.com/spring-expression-language-example-vs/>

OVERVIEW

Bean is an object that is instantiated and managed from creation to destruction by the **Spring IoC Container**.

Spring IoC Container is responsible for instantiating beans, wiring dependencies and managing the bean lifecycle from its instantiation till destruction. Spring have two IoC container implementations: **BeanFactory** and **ApplicationContext**.

Inversion of Control (IoC) is a process in which an object defines its dependencies without creating them explicitly. The object delegates the construction/instantiation job of its dependencies to the IoC container.

Dependency Injection is a pattern used to implement IoC, where the control being inverted is the setting and injection of object's dependencies.

SPRING EXPRESSION LANGUAGE

The Spring Expression Language (SpEL for short) is used to query property values from properties file or to manipulate Java objects and its attributes at runtime. **@Value** annotation is the most used way to process SpEL.

A SpEL expression begins with `#` and is enclosed by braces, e.g. `{2 + 2}`. Properties can be referenced in a similar fashion with `$`, and are enclosed by braces too, e.g. `{foo.bar}`. Property placeholders cannot contain SpEL expressions, but expressions can, e.g. `#{'${foo.bar}' + 2}`. Notice that to access the property contained in the properties file from SpEL, is mandatory to reference the property enclosed by single quotes.

SpEL provides two special built-in variables: `systemProperties` and `systemEnvironment`:

- **systemProperties** – a `java.util.Properties` object which provides runtime environment properties, like `os.name`, or JVM argument;
- **systemEnvironment** – a `java.util.Properties` object which provides access to environment variables;

STARTUP INTERFACES

Spring Boot provides two interfaces to run specific pieces of code as soon as the application starts: [CommandLineRunner](#) and [ApplicationRunner](#).

📌 When defining multiple beans of both types *in the same configuration class* ([@SpringBootApplication](#) or [@Configuration](#)), beans of [ApplicationRunner](#) type will be executed before beans of [CommandLineRunner](#) type. That rule also applies when they are defined individually as components ([@Component](#)). However when defined as components, that default execution order can be changed through the use of [@Order](#) annotation.

FACTORY HOOK INTERFACES

[BeanFactoryPostProcessor](#) - used to modify the definition of any bean before it get created/instantiated by working on its configuration metadata phase, such as loading value from external property files.

[BeanPostProcessor](#) - Factory hook interface that allows for custom modification of new bean instances *before* and *after* properties are set from initialization callbacks.

[InitializingBean](#) - interface to be implemented by beans that need to react once all their properties have been set by overriding the [afterPropertiesSet](#) method.

[DisposableBean](#) - interface to be implemented by beans that want to release resources upon their destruction by overriding the [destroy](#) method.

BEAN STATE ANNOTATIONS

[@PostConstruct](#) - used on beans to mark a method to be executed, like a callback, after the dependency injection is done.

[@PreDestroy](#) - used on beans to mark a method as a callback to signal the instance is in the process of being removed from the container;

CALLBACK ORDER EXECUTION

If all lifecycle callbacks, annotations and a BeanPostProcessor implementation are used in conjunction, the execution sequence goes like this:

- Bean's constructor
- BeanPostProcessor's `postProcessBeforeInitialization`
- `@PostConstruct` method
- InitializingBean's `afterPropertiesSet`
- Bean's `init`-method
- BeanPostProcessor's `postProcessAfterInitialization`
- `@PreDestroy` method
- DisposableBean's `destroy`
- Bean's `destroy`-method

KEY ANNOTATIONS

@Bean - used on factory methods to indicate that the produced object will be managed by the Spring container;

✦ If from a bean definition method another bean definition method is invoked, both within the same configuration class, and the called method is set to have `SINGLETON` scope then the returned object will always be the same, regardless of the number of invocations; i.e. even on direct calls, the scope defined for the bean determines whether the returning object has to be reused or a new one has to be created.

@Scope - used to indicate the scope for instances of the annotated bean;

@Autowired - used at constructors, methods and fields to indicate that the injection (i.e. instantiation) will be managed by the Spring container dynamically;

✦ **@Autowired** can not be used to inject primitive and string values. It only works with reference objects.

✦ **@Autowired**, when processing field injection, first looks for a bean which type is the same as the field to be injected. If multiple beans of the same type are found, then it may require further qualifications like the bean name or **@Qualifier** to determine the correct bean to be injected.

@Qualifier - used to specify the id of the bean that have to be used for injection when more than one type is eligible, i.e. when there are multiple beans resulting in ambiguity;

@Primary - used to indicate that a bean must be given preference when multiple candidates are qualified to autowire a single-valued dependency, i.e. is used to set higher preference for a given bean when there are multiple ones of the same type;

📌 When the injection of a field/parameter marked to be autowired is going to be via its type and there are many candidate instances for the given type, an `NoUniqueBeanDefinitionException` will be thrown. To solve, we can either add `@Primary` to one of the bean definition or add `@Qualifier` to the field/parameter specifying the name from one of the matching candidate beans

`@Lazy` - used to indicate whether a bean is to be lazily initialized, i.e. if present on a `@Component` or `@Bean` definition and set to `true`, the bean or component will not be initialized until referenced by another bean or explicitly retrieved from the enclosing `BeanFactory`. This annotation may also be placed on injection points marked with `@Autowired`, like constructor parameters, method parameters etc;

`@Profile` - used to indicate that a component class or bean is eligible for registration when the specified profile(s) are active;

`@Component` - generic stereotype annotation used to indicate that a class is a managed component, registering as a bean;

`@Repository` - is a `@Component` stereotype annotation used to indicate that a class defines a data repository. **Enables automatic translation of exceptions thrown from the underlying persistence layer;**

`@Service` - is a `@Component` stereotype annotation used to indicate that a class defines business logic;

`@Configuration` - **used to mark a class as a source of bean definitions.**

📌 Spring creates dynamic proxies for classes annotated with `@Configuration` and uses CGLIB to extend those class to create proxies. Hence, `@Configuration` classes must not be final, because final classes cannot be extended, i.e. cannot be subclassed;

`@Import` - used to indicate one or more component class to import — typically from `@Configuration` classes.

`@ImportAutoConfiguration` - used to disable the default autoconfiguration flow performed by `@EnableAutoConfiguration`, importing only the configuration classes provided in the annotation.

`@Value` - **used at field or method/constructor parameter level to inject value from properties located at .properties/.yml files, SpEL, classpath's resources etc;**

`@PropertySource` - used to load values from property files — the values can be accessed from `Environment` or injected by the `@Value` annotation;

`@ConfigurationProperties` - used to map resource files such as properties or YAML files to Java Bean object.

[@EnableConfigurationProperties](#) - enables support for `@ConfigurationProperties` annotated classes in our application.

5. SPRING BOOT BASICS AND AUTO-CONFIGURATION

SAMPLE PROJECTS

- [spring-conditional](#)

REFERENCES

- <https://www.baeldung.com/spring-boot-annotations>
- <https://www.baeldung.com/spring-conditional-annotations>
- <https://docs.spring.io/spring-boot/docs/2.0.x/reference/html/using-boot-auto-configuration.html>
- <https://www.marcobehler.com/guides/spring-boot>
- <https://www.baeldung.com/spring-component-scanning>
- <https://reflectoring.io/spring-boot-conditionals>
- <https://zetcode.com/springboot/conditionalbeans>
- <https://www.baeldung.com/spring-boot-custom-starter>

AUTO-CONFIGURATION

Auto-configuration is a mechanism in which Spring automatically configures an application based on the dependencies found on its classpath.

When every Spring application boots up, Spring tries to read in properties from 17 locations in addition to the `spring.factories` file in the classpath.

`spring.factories` file is a special file that Spring automatically loads when booting up. It contains reference to a lot of `@Configuration` class mappings which Spring will try to run. The `spring.factories` file is located in `META-INF` of the [org.springframework.boot:spring-boot-autoconfigure](#) dependency.

@Conditional-based annotations are one of the key pieces that makes Spring auto-configuration mechanism work. We can apply [@Conditional-based annotations to common Spring components, like @Bean, @Component, @Service, @Repository and @Controller.](#)

[@Conditional](#) - used to indicate that a given component is only eligible for registration when all specified condition match;

[@ConditionalOnBean](#) - used to condition the registration of the annotated component when beans of all classes specified are contained in the BeanFactory;

[@ConditionalOnMissingBean](#) - used to condition the registration of the annotated component when none of the bean class specified is contained in the BeanFactory;

[@ConditionalOnClass](#) used to condition the registration of the annotated component only if the specified classes are on the classpath;

[@ConditionalOnMissingClass](#) - used to condition the registration of the annotated component only if none of the specified classes are on the classpath;

[@ConditionalOnExpression](#) - used to condition the registration of the annotated component only if the specified SpEL expression returns true;

[@ConditionalOnProperty](#) - used to condition the registration of the annotated component only if the specified property is set;

[@ConditionalOnResource](#) - used to condition the registration of the annotated component only if the specified resources exist;

[@ConditionalOnJava](#) - used to condition the registration of the annotated component only if the application is running on the specified JVM version. By default returns true if the running JVM version is equal to or greater than the specified version;

[@ConditionalOnJndi](#) - used to condition the registration of the annotated component only if the specified JNDI context exists;

[@ConditionalOnWebApplication](#) - used to condition the registration of the annotated component only if the application is a web application (SERVLET or REACTIVE);

[@ConditionalOnNotWebApplication](#) - used to condition the registration of the annotated component only if the application is not a web application;

[@ConditionalOnWarDeployment](#) - used to condition the registration of the annotated component only if the application is a traditional WAR deployment. For applications with embedded servers, this condition will always return `false`.

[@ConditionalOnSingleCandidate](#) - used to condition the registration of the annotated component only if a bean of the specified class is already contained in the BeanFactory and only a single candidate can be determined;

[@ConditionalOnCloudPlatform](#) - used to condition the registration of the annotated component only if the specified cloud platform is active;

CUSTOM CONDITION

We can define a custom logic to be used as criteria for registering a component. To do so we should create a class that implements the [Condition](#) interface, overrides its [matches](#) method with our custom logic, and then specify our class as parameter for the `@Conditional` annotation.

COMBINE CONDITIONS

We can combine `@Conditional`-based annotations along with custom conditions in order to implement complex OR or AND logical operation.

To apply OR operation, we have to create a custom condition extending the [AnyNestedCondition](#) class, then uses it as argument in the `@Conditional` annotation. For that purpose I created the [HmlOrPrdEnvironmentCondition](#) custom condition and applied it [here](#).

To apply AND operation, we can group custom conditions in the `@Conditional` and additionally set others `@Conditional`-based annotations to the component. I demonstrate this operation [here](#).

External Application Properties

By default, Spring Boot will find and load `application.[properties|yaml]` files from the following locations when your application boots up. This list is ordered by precedence with values from lower items overriding earlier ones:

✚ Spring applications have in its classpath the `src/main/resource` folder location by default.

✚ If we have the `application.properties` and `application.yml` files in the same location and a given property is defined in both, the value from `application.properties` will take precedence over the value at `application.yml`.

✚ Command line properties, JVM arguments and OS environment variables will always take precedence over (same) properties defined from `application.properties` or `application.yml`.

1. From command-line and OS environment:

- OS environment variable, e.g. `export server.port=9090 ;`
- JVM argument, e.g. `-Dserver.port=9090 ;`
- command-line argument, e.g. `--server-port=9090 ;`

2. From the classpath:

- The classpath root, e.g. `/resources` folder by default;
- The classpath `/config` folder, i.e. `/resources/config` ;

3. From the current directory:

- The current directory;
- The `/config` subdirectory in the current directory;
- Immediate child directories of the `/config` subdirectory, according to system default folder ordering;

With that in mind, if a property is defined in `/resources/application.properties` and there is a definition for the same property in `/resources/config/application.properties` with a different value, the value from the latter location will take precedence over the former.

KEY ANNOTATIONS

`@SpringBootApplication` - is a combination of `@Configuration`, `@EnableAutoConfiguration`, and `@ComponentScan` annotations with their default attributes;

`@EnableAutoConfiguration` – used to indicate to `ApplicationContext` to add beans **based on the dependencies on the classpath automatically**;

`@ComponentScan` – used to indicate to `ApplicationContext` to scan for other components, **configurations and beans in the same package as the Application class and below**;

`@SpringBootConfiguration` - used to indicate that a class provides Spring Boot application `@Configuration`. Can be used as an alternative to `@Configuration` annotation so that configuration can be found automatically;

6. ASPECT-ORIENTED PROGRAMMING

SAMPLE PROJECTS

- `spring-aop`

REFERENCES

- <https://www.javatpoint.com/spring-aop-tutorial>
- <https://docs.spring.io/spring-framework/docs/3.0.x/reference/aop.html>
- <https://docs.spring.io/spring-framework/docs/current/reference/html/core.html#aop>
- <https://www.journaldev.com/2583/spring-aop-example-tutorial-aspect-advice-pointcut-joinpoint-annotations>
- <https://howtodoinjava.com/spring-aop-tutorial/>
- <https://www.baeldung.com/spring-aop-pointcut-tutorial>
- <https://www.tutorialandexample.com/spring-aop-pointcut-expressions/>
- <https://www.eclipse.org/aspectj/doc/next/aspectj5rt-api/org/aspectj/lang/annotation/package-summary.html>

OVERVIEW

Aspect-Oriented Programming (AOP) complements Object-Oriented Programming (OOP) by providing another way of thinking about program structure. The key unit of modularity in OOP is the *class* whereas in AOP the unit of modularity is the *aspect*.

CORE CONCEPTS

- **Cross-Cutting Concerns** - are common functions that span on multiple points of an application, such as logging, transaction management, data validation, etc;
- **Join Point** - is any point during the execution of a program, such as a method execution, an exception handling or a field access. **In Spring AOP, a join point always represents a method execution;**
- **Pointcut** - is an expression language from AOP that matches **Joint Point**. Spring uses the AspectJ pointcut expression language by default;
- **Advice** - is an action(s) taken by an **Aspect** at a particular **Join Point**;
- **Aspect** - **is a class that contains Advices, Join Points etc;**
- **Introduction** - is a means to declare additional methods and fields for a type, allowing to introduce new interface to any advised object;
- **Target Object** - is the object being advised by one or more **Aspects**, i.e. the object on which advices are applied. It is known as proxied objects in Spring;
- **Interceptor** - is an Aspect that contains only one advice;

- **AOP Proxy** - is an object created by the AOP framework in order to implement the aspect contracts. In Spring, an AOP proxy will be a *proxied object*;
- **Weaving** - is the process of linking aspects with other application types or objects to create an advised object. Weaving can be done at compile time, load time or runtime. Spring AOP performs weaving at runtime;

KEY INTERFACE

JoinPoint - provides reflective access to both the state available at a join point and static information about it.

ProceedingJoinPoint - extends from JoinPoint and exposes the `proceed(..)` method in order to support `@Around` advice.

KEY ANNOTATIONS

@Aspect - used to mark a `@Component` class as an aspect declaration;

@Pointcut - used in methods to declare pointcut, which can be availed by advice annotations on referring to that pointcut;

@Before - used to mark a method to be executed before the matching joint point;

@After - used to mark a method to be executed after the matching join point finishes, whether normally or by throwing an exception;

@AfterReturning - used to mark a method to be executed after (only if) the matching joint point completes normally without throwing exception;

@AfterThrowing - used to mark a method to be executed after (only if) the matching join point exits due to an exception;

@Around - used to mark a method to be executed before and after a Joint Point.

7. DATA MANAGEMENT: JDBC, TRANSACTIONS

SAMPLE PROJECTS

- [spring-datasource](#)
- [spring-jdbctemplate](#)
- [spring-transaction](#)

REFERENCES

- <https://www.javainuse.com/spring/boot-transaction>
- <https://www.baeldung.com/spring-transactional-propagation-isolation>
- <https://www.marcobehler.com/guides/spring-transaction-management-transactional-in-depth>
- <https://www.byteslounge.com/tutorials/spring-transaction-propagation-tutorial>
- <https://www.byteslounge.com/tutorials/spring-transaction-isolation-tutorial>

TRANSACTIONS

Transaction is a single logical unit of work which could be composed by one or many actions that potentially modify the content of a database, i.e. a sequence of actions that are considered as a single logical unit by the application. For an application, if any action running into a transactional context fails then all other actions gets rolled back. **In Spring, database transaction is by default `auto-commit`**, i.e. every SQL statement runs in its own transaction and will commit after execution. **With `@Transactional` in place, by default Spring will only rolls back on *unchecked exceptions* (*RuntimeException and its subclasses*).** To make a transaction rolls back on checked exception we have to specify it on the `@Transaction's rollbackFor` parameter.

Transaction Propagation is a mechanism used to indicate **if a given action** will or will not participate in a transactional context as well as how it will behave when called from a context which already has or not a transaction in place.



The default transaction propagation type is **REQUIRED**



Read-only transaction is a **transaction which does not modify any data**. If we use the `@Transactional's readOnly` attribute set to `true` on a method which is performing create or update operation then we will not have any created or updated record into the database but rather an exception



If we have many `@Transaction` methods in the same class defined with different propagation types and if they are called sequentially, in practice only the propagation configuration of the first method in the flow will be considered. That is because Spring creates a proxy upon detecting the `@Transactional` annotation when we are calling any internal method, it will bypass the proxy

📌 If we manually handle exceptions (including unchecked exceptions) via try-catch from a transactional context and an exception pops up, the rollback mechanism won't work and the current transaction will execute and commit normally

📌 To make transaction suspension works across nested method calls, the methods must belong to different object instances

📌 The @Transactional annotation at the class level will be applied to all of its public method. However, if a method is annotated with @Transactional its settings will overwrite the transactional settings defined at the class level

TYPES OF TRANSACTION PROPAGATION

Propagation	Behaviour
REQUIRED	Always executes in a transaction. If there is an active transaction it is used otherwise a new one is created
REQUIRES_NEW	Always executes in a new transaction. If there is an active transaction it gets suspended before stars a new one
NESTED	Participates in the existing transaction by creating <i>subtransaction</i> which sets <i>savepoints</i> between nested method invocations so that these subtransactions can roll back independently of the outer transaction upon failure
SUPPORTS	If a physical transaction exists then it makes use otherwise the method will continue its execution in a non-transactional context
NOT_SUPPORTED	Always executes outside of a transactional context. If there is a transaction in place it gets suspended until the end of the method execution
NEVER	Must execute outside of a transactional context. If there is an active transaction an <code>IllegalTransactionStateException</code> is thrown
MANDATORY	Must execute inside a transactional context. If no active transaction is found an <code>IllegalTransactionStateException</code> is thrown

📌 If we try to use `NESTED` propagation with Hibernate JPA we will get a `NestedTransactionNotSupportedException` exception with the following message:
 JpaDialect does not support savepoints - check your JPA provider capabilities. That happens because of the Hibernate JPA implementation which does not support nested transactions. To solve, we can use `JdbcTemplate` or a JPA provider that supports nested transactions, or use `JOOQ`.

Transaction Isolation is a mechanism used to indicate how changes made to data by one transaction affect other concurrent transactions as well as how and when changed data becomes available to other concurrent transactions, i.e. how changes applied on data by concurrent transactions are visible to each other.

TYPES OF TRANSACTION ISOLATION

Isolation	Behaviour
DEFAULT	Indicates that for any transaction the default isolation level of the underlying RDBMS will be used
READ_COMMITTED	Indicates that a transaction can only read data that is committed by concurrent transactions
READ_UNCOMMITTED	Indicates that a transaction may read data that is still uncommitted by concurrent transactions
REPEATABLE_READ	Indicates that if a transaction reads one record from the database multiple times the result of all reading must always be the same
SERIALIZABLE	Indicates that transactions must be executed with locking at all levels (read, range and write locking) so that they behave as if they were executed in a serialized way, i.e. concurrent execution of a group of transactions should have the same result as if they are executed sequentially



In Spring with JPA if we try to use an isolation level that is different from the DEFAULT isolation type which the transaction manager implementation does not support, we will get an `InvalidIsolationLevelException` exception with the following message: `Standard JPA does not support custom isolation levels - use a special JpaDialect for your JPA implementation`. One way to solve this problem is implement a custom JPA dialect.

KEY INTERFACES

ResultSet - is used to access data produced by the execution of database queries. It provides getter methods (getBoolean, getLong, and so on) for retrieving column values (by name or column's index) from the current row.

RowMapper - is used by JdbcTemplate for mapping rows of a ResultSet to Java object *on a per-row basis*.

KEY ANNOTATIONS

[@Transactional](#) - used to indicate declaratively control over transaction boundaries on managed beans, usually for [@Service](#) classes which contain business logic;

[@EnableTransactionManagement](#) - used on [@Configuration](#) classes to enable the Spring's annotation-driven declarative transaction management capabilities. This annotation is **optional** in Spring Boot application as long as one of `spring-data-*` modules is configured on the project;

8. SPRING DATA JPA

SAMPLE PROJECTS

- [spring-data-jpa](#)

REFERENCES

- <https://spring.io/guides/gs/accessing-data-jpa/>
- <https://knppcode.com/spring/spring-data-tutorial/>
- <https://www.baeldung.com/the-persistence-layer-with-spring-data-jpa>
- <https://www.baeldung.com/spring-data-jpa-query>

OVERVIEW

Spring Boot configures Hibernate as the default JPA provider, and auto-configures a `DataSource` bean if there's in-memory database dependency of type H2, HSQLDB or Derby present on the classpath. Spring Boot applications may be configured with multiple databases of different types.

Spring Boot allows to configure `DataSource` in two ways: programmatically via a [@Configuration](#) class or from properties configuration file.

KEY INTERFACES

[CrudRepository](#) - provides CRUD operations on a repository

[PagingAndSortingRepository](#) - extends from `CrudRepository` and provides methods for pagination and sorting records

JpaRepository - extends from PagingAndSortingRepository and additionally provides JPA related methods such as flushing the persistence context and delete records in a batch

TESTING

By default, transactions in JUnit tests are flagged for rollback when they start. However, if the method is annotated with `@Commit` or `@Rollback(false)`, they start flagged for commit instead.

By default `@DataJpaTest` autoconfigures an in-memory database like H2 to be used on testing, as long as one of that type is found in the classpath. To avoid start an embedded database and use a real one like Postgres we must annotate the test class with

```
@AutoConfigureTestDatabase(replace = AutoConfigureTestDatabase.Replace.NONE) .
```

KEY ANNOTATIONS

@Entity - used to specify that the annotated class **maps a database entity**. If we forget to mark a domain class that will map a database entity with `@Entity` annotation, we will get an `IllegalArgumentException` since Spring will not be able to invoke the init method in a not managed type;

@Table - used to specify the table's details, like name, schema etc, that will map the entity in the database. **It's not mandatory to use @Table on mapping an entity class, and in its absence, the table name will be taken from the class name;**

@Id - used to mark a property in a entity class as the primary key. If no `@Column` annotation is specified, the primary key column name is assumed to be the name of the property;

@GeneratedValue - used to specify the primary key generation strategy which by default is autoincrement;

@Column - used to specify database column details like name, constraint etc to which a property will be mapped. **If no @Column annotation is specified, then the field name will be used for mapping;**

@Query - used to declare custom queries on **repository methods** to execute both JPQL or native SQL queries;

@Modifying - used to indicate that a **@Query annotated repository method** executes a modifying JPQL or native SQL such as a INSERT, UPDATE, DELETE or DDL statement;

@EntityScan - used on `@Configuration` classes to indicate to Spring where is located entity classes when they are not placed under the main application package or its sub-packages;

[@EnableJpaRepositories](#) - used on `@Configuration` classes to indicate to Spring where is located JPA repository classes when they are not placed in the same package of the main application class or under its sub-packages;

9. SPRING MVC

SAMPLE PROJECTS

- [spring-crud](#)
- [spring-mvc](#)
- [spring-web](#)

REFERENCES

- <https://www.marcobehler.com/guides/spring-mvc>
- <https://www.baeldung.com/spring-mvc-tutorial>
- <https://spring.io/guides/gs/serving-web-content>
- <https://docs.spring.io/spring-framework/docs/3.2.x/spring-framework-reference/html/mvc.html>
- <https://howtodoinjava.com/spring-boot2/resttemplate/spring-restful-client-resttemplate-example>

KEY INTERFACES

[Model](#) - used to pass values required for rendering views;

[ModelMap](#) - used to pass values to render a view, with the advantage to pass a collection of values like a Map.

[ModelAndView](#) - used to pass all the information, i.e. model and view, required to process page rendering;

KEY ANNOTATIONS

[@ModelAttribute](#) - used to bind method parameter or method return value to a named model attribute, exposed in a web view.

[@GetMapping](#) - used to map HTTP GET requests onto specific handler method. It is a shortcut for `@RequestMapping(method = RequestMethod.GET)` ;

@PostMapping - used to map HTTP POST requests onto specific handler method. It is a shortcut for `@RequestMapping(method = RequestMethod.POST)` ;

@PutMapping - used to map HTTP PUT requests onto specific handler method. It is a shortcut for `@RequestMapping(method = RequestMethod.PUT)` ;

@DeleteMapping - used to map HTTP DELETE requests onto specific handler method. It is a shortcut for `@RequestMapping(method = RequestMethod.DELETE)` ;

@PatchMapping - used to map HTTP PATCH requests onto specific handler method. It is a shortcut for `@RequestMapping(method = RequestMethod.PATCH)` ;

@RequestMapping - used to map web requests onto methods in request-handling classes with flexible method signatures. `@GetMapping`, `@PostMapping`, `@PutMapping`, `@DeleteMapping`, and `@PatchMapping` are different variants of `@RequestMapping` for handling the specific HTTP operation by these annotations. By default `@RequestMapping` maps all HTTP operations;

@PathVariable - used to bind a named URI path parameter into a method argument;

@RequestParam - used to bind a URI query parameter to a method argument;

@RequestHeader - used to bind a request header to a method parameter;

@RequestPart - used to associate a part of a multipart request with a method argument;

@CookieValue - used to indicate that a method parameter is bound to an HTTP cookie;

@RequestBody - used to bind the payload body from the request onto a method argument;

@ResponseBody - used to indicate that the result of a request handler method is the response itself. If a class is annotated with `@ResponseBody`, all of its request handler methods will be functioning like that;

@ResponseStatus - used to specify the HTTP response status of a request handler method;

@Controller - is a `@Component` stereotype annotation used to mark a class as a source of request method handlers;

@RestController - combines the `@Controller` and `@ResponseBody` annotations, making all request handler methods in the annotated class assumes `@ResponseBody` by default;

✦ We can assign URI template variables or query parameters directly to objects without having to specify the `@PathVariable` and `@RequestParam` annotations, respectively; as long as the given object has matching getters/setters

✦ `@RequestMapping`'s `headers` and `params` attributes are used to narrow requests that contain specific headers or query parameters, respectively

📌 @RequestMapping methods mapped to "GET" are also implicitly mapped to "HEAD", i.e. there is no need to have "HEAD" explicitly declared. An HTTP HEAD request is processed as if it were an HTTP GET except instead of writing the body only the number of bytes are counted and the "Content-Length" header set.

📌 @RequestMapping methods have built-in support for HTTP OPTIONS. By default an HTTP OPTIONS request is handled by setting the "Allow" response header to the HTTP methods explicitly declared on all @RequestMapping methods with matching URL patterns. When no HTTP methods are explicitly declared the "Allow" header is set to "GET,HEAD,POST,PUT,PATCH,DELETE,OPTIONS"

📌 If @RequestMapping is used along with any of its variant annotations in a handler method definition, its configuration will take precedence over the descendant annotation, i.e. only the @RequestMapping settings (e.g. [method](#), [path](#), [query parameters](#), [headers](#)) will be taken into account.

10. SPRING SECURITY

REFERENCES

- <https://www.toptal.com/spring/spring-security-tutorial>
- <https://spring.io/guides/topicals/spring-security-architecture>
- <https://www.marcobehler.com/guides/spring-security>
- <https://www.baeldung.com/spring-security-expressions>
- <https://www.section.io/engineering-education/springboot-antmatchers>
- <https://levelup.gitconnected.com/learn-how-to-build-a-token-based-authentication-server-using-spring-boot-and-spring-security-14a82d186f88>
- <https://medium.com/wolox/securing-applications-with-jwt-spring-boot-da24d3d98f83>
- <https://medium.com/geekculture/spring-security-authentication-process-authentication-flow-behind-the-scenes-d56da63f04fa>

OVERVIEW

Spring Security by default activates both **HTTP security filters and the Security Filter Chain** and configures/enforces session-based basic authentication for all URLs.

The Spring Security framework provides two options to set up authorization schema configuration: URL-based and Annotation-based.

- **AuthenticationManager** - works like a coordinator where authentication providers are registered.
- **AuthenticationProvider** - processes specific types of authentication.
- **UserDetailsService** - core interface that loads user-specific data in the Spring Security flow.
- **antMatcher()** - method used to configure URL access restrictions by using **ant patterns** so that they are only invoked if the given pattern matches.
- **mvcMatcher()** - serves for the same purpose as antMatcher but is designed to apply precisely to what the Spring MVC `@RequestMapping` and related annotations understand, e.g. `"mvcMatcher(/hello)"` will also match `/hello` and `/hello/`, while with `antMatcher()` only the former will match.

CORE CONCEPTS

- **Authentication** - refers to the process of verifying the identity of a user, based on provided credentials. A common example is entering a username and a password when you log in to a website. You can think of it as an answer to the question: *Who are you?*
- **Authorization** - refers to the process of determining if a user has proper permission to perform a particular action or access particular data, assuming that the user is successfully authenticated. You can think of it as an answer to the question: *Can a user do/access this?*
- **Principle** - refers to the currently authenticated user.
- **Granted Authority** - refers to the permission of the authenticated user.
- **Role** - refers to a group of permissions which the authenticated user have.

NOTES

- Double wildcard pattern (`**`) will match **the current path (directory) and below**. For instance, the following rule will match `api/public`, `api/public/users`, `api/public/users/1` and so forth:

```
.mvcMatchers("/api/public/**")
```

- Single wildcard pattern (`*`) will match **the path under the specified directory and below**. For instance, the following rule will match `api/public/users`, `api/public/users/1` and so forth:


```
.mvcMatchers("/api/public/*")
```

- URL pattern definitions with role access association should be defined from most to less specific. For instance, in the following definition the pattern `/api/public/get` should be put ideally before `/api/public/**`:

```
.mvcMatchers("/api/public/**").hasRole(Roles.USER_ADMIN)  
.mvcMatchers("/api/public/get").hasRole(Roles.AUTHOR_ADMIN)
```

PATTERN DEFINITION SCENARIOS

- Scenario #1

Logged as: AUTHOR_ADMIN

Pattern:

```
.antMatchers("/api/public/get").hasRole(Roles.AUTHOR_ADMIN)  
.antMatchers("/api/public/**").hasRole(Roles.USER_ADMIN)
```

Path	Status
<code>/api/public</code>	403
<code>/api/public/get</code>	200

- Scenario #2

Logged as: AUTHOR_ADMIN

Pattern:

```
.antMatchers("/api/public/**").hasRole(Roles.USER_ADMIN)  
.antMatchers("/api/public/get").hasRole(Roles.AUTHOR_ADMIN)
```

Path	Status
<code>/api/public</code>	403
<code>/api/public/get</code>	403

- Scenario #3

Logged as: AUTHOR_ADMIN

Pattern:

```
.antMatchers("/api/public/*").hasRole(Roles.USER_ADMIN)
.antMatchers("/api/public/get").hasRole(Roles.AUTHOR_ADMIN)
```

Path	Status
/api/public	200
/api/public/get	403

- Scenario #4

Logged as: AUTHOR_ADMIN

Pattern:

```
.antMatchers("/api/public/get").hasRole(Roles.AUTHOR_ADMIN)
.antMatchers("/api/public/*").hasRole(Roles.USER_ADMIN)
```

Path	Status
/api/public	200
/api/public/get	200

KEY ANNOTATIONS

[@EnableWebSecurity](#) - marks a `@Configuration` class as a source of web access security configuration. Usually such class extends the [WebSecurityConfigurerAdapter](#) class and overrides its methods for a more granular configuration;

[@EnableGlobalMethodSecurity](#) / [@EnableMethodSecurity](#) - both are used to configure security on method level through annotations. They have the following attributes:

1. `securedEnabled`

If set to true enables `@Secured` annotation. Default is false

2. `jsr250Enabled`

If set to true enables `@RolesAllowed` annotation. Default is false

3. `prePostEnabled`

If set to true enables `@PreAuthorize`, `@PostAuthorize`, `@PreFilter`, `@PostFilter` annotations. Default is false

[@PreAuthorize](#) - supports SpEL and is used to provide expression-based access control before executing the method;

[@PostAuthorize](#) - supports SpEL and is used to provide expression-based access control after executing the method and provides the ability to access/alter the method's result;

[@PreFilter](#) - supports SpEL and is used to filter the collection or arrays before executing the method based on custom security rules we define;

[@PostFilter](#) - supports SpEL and is used to filter the returned collection or arrays after executing the method based on custom security rules we define (provides the ability to access the method result);

[@Secured](#) - does not support SpEL and is used to specify a list of roles which the logged user must have in order to access the annotated method;

[@RolesAllowed](#) - does not support SpEL and is the [JSR-250](#)'s equivalent annotation of the [@Secured](#) annotation;

[@WithMockUser](#) - can be added to a test method to emulate running with a mocked user;

[@WithUserDetails](#) - can be added to a test to emulate a UserDetails instance which is returned from the UserDetailsService according to the given username;

[@WithAnonymousUser](#) - can be added to a test method to emulate running with an anonymous user. This is useful when a user wants to run a majority of tests as a specific user and override a few methods to be anonymous;

[@WithSecurityContext](#) - determines what SecurityContext to use, and all three annotations described above are based on it. If we have a specific use case, we can create our own annotation that uses [@WithSecurityContext](#) to create any SecurityContext we want;

11. SPRING BOOT ACTUATOR

SAMPLE PROJECTS

- [spring-actuator](#)

REQUIRIMENT TO USE THE SAMPLE PROJECTS

- PostgreSQL



To spin up a PostgreSQL instance by Docker, you can use the following command:

```
docker run --name postgres-14.1 -e POSTGRES_USER=postgres -e  
POSTGRES_PASSWORD=postgres -p 5432:5432 --rm -it postgres:14.1-alpine
```

REFERENCES

- <https://www.baeldung.com/spring-actuators>
- <https://www.baeldung.com/spring-boot-health-indicators>
- <https://docs.spring.io/spring-boot/docs/current/reference/html/actuator.html#>
- <https://lightrun.com/best-practices/getting-started-with-spring-actuator>
- <https://www.javadevjournal.com/spring-boot/spring-actuator-custom-endpoint>
- <https://dimitr.im/mastering-spring-boot-actuator>

OVERVIEW

Spring Boot Actuator provides resources so we can monitor and manage our application's health and availability. It is mainly used to expose operational information about the running application, such as health, metrics, info, dump, environment, loggers etc.

BUILT-IN ENDPOINTS

By default, Spring Boot Actuator comes with all built-in endpoints disabled, except `/health`. A given endpoint is considered *available* when it is both *enabled* and *exposed*. Spring Boot Actuator's endpoints are exposed over REST endpoints and JMX.

Table of built-in endpoints provided out of the box:

ENDPOINT	DESCRIPTION
<code>/auditevents</code>	Returns audit event information for the current application
<code>/beans</code>	Returns a complete list of all Spring beans in the current application
<code>/caches</code>	Returns available caches
<code>/conditions</code>	Returns a report of all conditions around autoconfiguration
<code>/configprops</code>	Returns a grouped list of all <code>@ConfigurationProperties</code>
<code>/env</code>	Returns the current environment properties from Spring's <code>ConfigurableEnvironment</code>
<code>/health</code>	Returns application health information
<code>/httptrace</code>	Returns HTTP trace informations (last 100 HTTP request-response exchanges by default)
<code>/info</code>	Returns arbitrary application information

ENDPOINT	DESCRIPTION
<code>/integrationgraph</code>	Returns the Spring integration graph. Requires a spring-integration-core dependency
<code>/loggers</code>	Returns the configuration of all loggers in the application. Can be used to modify at run time a logger level
<code>/metrics</code>	Returns metrics information for the current app
<code>/mappings</code>	Returns a grouped list of all your application's APIs
<code>/scheduledtasks</code>	Returns the tasks scheduled in your application
<code>/sessions</code>	Returns retrieval and deletion of user sessions from a Spring Session supported session store. Requires a Servlet-based web application using Spring Session
<code>/shutdown</code>	Disables the application. It is not enabled by default (To enable <code>management.endpoint.shutdown.enabled=true</code>)
<code>/startup</code>	Returns the startup step data collected by ApplicationStartup
<code>/threaddump</code>	Performs a thread dump

INFO ENDPOINT

To expose custom information on `/actuator/info` we should set them as property entries starting with `info.*` at `application.yml` or `application.properties` . Also, it's required to set `management.info.env.enabled` to `true` because the Spring Actuator's `InfoContributor` is disabled by default.



The `/info` endpoint comes disabled by default. To enable it we have to set the property `management.endpoints.web.exposure.include` including it as entry.

GIT & BUILD INFORMATION

To get git and build details returned on `/actuator/info` , we have to add the following to the plugin section:

```
<plugin>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-maven-plugin</artifactId>
  <executions>
    <execution>
      <goals>
        <goal>build-info</goal>
      </goals>
    </execution>
  </executions>
</plugin>
```

```
        </goals>
      </execution>
    </executions>
  </plugin>
  <plugin>
    <groupId>p1.project13.maven</groupId>
    <artifactId>git-commit-id-plugin</artifactId>
  </plugin>
```

HEALTH ENDPOINT

HEALTH INDICATOR

Health indicators can be used to report the healthiness and availability of inner components as well as third-party services required for the application. Spring Boot Actuator comes with predefined health indicators which provide specific status. Such built-in indicators are: [DiskSpaceHealthIndicator](#), [PingHealthIndicator](#), [LivenessStateHealthIndicator](#) and [ReadinessStateHealthIndicator](#). Some health indicators are registered and added automatically according to the existence of specific dependencies on the classpath or when other conditions are satisfied. For instance, [\[DataSourceHealthIndicator\]\(https://docs.spring.io/spring-boot/docs/current/api/org/springframework/boot/actuate/jdbc/DataSourceHealthIndicator.html\)](#) will be available if any relational database driver dependency is found; [RedisHealthIndicator](#) will be available if the Redis dependency module is found on the classpath.

Table of Health Indicators provided out of the box:

HEALTH INDICATOR	ENDPOINT
CassandraHealthIndicator	<code>/actuator/health/cassandra</code>
CouchbaseHealthIndicator	<code>/actuator/health/couchbase</code>
DataSourceHealthIndicator	<code>/actuator/health/db</code>
DiskSpaceHealthIndicator	<code>/actuator/health/diskSpace</code>
ElasticsearchRestHealthIndicator	<code>/actuator/health/elasticsearch</code>
HazelcastHealthIndicator	<code>/actuator/health/hazelcast</code>
InfluxDbHealthIndicator	<code>/actuator/health/influx</code>
JmsHealthIndicator	<code>/actuator/health/jms</code>
LdapHealthIndicator	<code>/actuator/health/ldap</code>
LivenessStateHealthIndicator	<code>/actuator/health/liveliness</code>

HEALTH INDICATOR	ENDPOINT
MailHealthIndicator	/actuator/health/mail
MongoHealthIndicator	/actuator/health/mongo
Neo4jHealthIndicator	/actuator/health/neo4j
PingHealthIndicator	/actuator/health/ping
RabbitHealthIndicator	/actuator/health/rabbit
ReadinessStateHealthIndicator	/actuator/health/readiness
RedisHealthIndicator	/actuator/health/readiness
SolrHealthIndicator	/actuator/health/solr

The Liveness and Readiness HTTP probe statuses are returned by default on `/actuator/health` for applications running on Kubernetes. To enable them to be exposed otherwise we have to set the property `management.health.probes.enabled` to `true`.

By default `/actuator/health` does not return detailed informations about the available indicators. To get them exposed, we have to set both properties `management.endpoint.health.show-components` and `management.endpoint.health.show-details` to `always`.

By default there are four types of health status: `UP`, `DOWN`, `OUT_OF_SERVICE` and `UNKNOWN`, and for each there is a corresponding associated HTTP code.

STATUS	HTTP_CODE	MEANING
UP	200	The component or subsystem is functioning as expected
DOWN	503	The component or subsystem has suffered an unexpected failure
OUT_OF_SERVICE	503	The component or subsystem has been taken out of service and should not be used
UNKNOWN	200	The component or subsystem is in an unknown state

The application's **overall status** is an aggregation of all health indicators statuses, i.e. from all built-in (db, diskSpace, ping, livenessState, readiness etc) and custom health indicators. If one of them is in failure or unknown state, *the root status will be reported as DOWN*.

CUSTOM HEALTH INDICATORS

To register a custom health indicator we have to create a `@Component` class that implements the `HealthIndicator` interface and overrides the `health()` method.

To generate a composite health check indicator in order to combine many indicators we have to create a `@Component` class that implements the `CompositeHealthContributor` interface; mark each of the contributing health indicators with the `HealthContributor` interface; and override the `iterator()` method in the `CompositeHealthContributor` implementation component class with the list of health contributors (component class that implements `HealthContributor`)

METRICS

The Spring Actuator exposes the `/actuator/metrics` endpoint which returns performance and operational measurements. To collect and gather metrics Spring Actuator relies on the `Micrometer` framework, which acts as a facade – an intermediate layer – between the application and some of the more popular monitoring tools, like Prometheus, Elastic, DataDog, Dynatrace etc.

METRIC TYPES

COUNTERS

A `Counter` represents a value that will always get incremented. It is applicable to measure a number of events or actions that is supposed to only increase, and never decreases.

GAUGES

A `Gauge` represents a value that may go up or down, and that have fixed upper bounds.

TIMERS

A `Timer` is used to track how long certain method, action or request takes to execute.

☰ README.md

Spring Actuator provides the `/actuator/loggers` endpoint which returns all the loggers, and their configured log levels, existing in the application. To check details for an individual logger we access the endpoint `http://localhost:8081/actuator/loggers/{logger.name}`, where `{logger.name}` should be the name of the desired logger. For instance, `http://localhost:8081/actuator/loggers/io.davidarchanjo` would output the following:

```
{
  "configuredLevel": "null",
```



```
"effectiveLevel": "INFO"  
}
```

CHANGING LOGGING LEVEL

Spring Actuator also provides a way to change the current log level at run time. To change we issue a POST request specifying the logger with the desired new log level. For instance, to set the log level to `TRACE` of the package `io.davidarchanjo` the request goes as below:

```
$ curl -X POST \  
  localhost:8081/actuator/loggers/io.davidarchanjo \  
  -H 'content-type: application/json' \  
  -d '{"configuredLevel":"TRACE"}'
```

KEY ANNOTATIONS

[@Endpoint](#) - used to indicate a type as being an actuator endpoint that provides information about the running application;

[@RestControllerEndpoint](#) - used to indicate a type as being a REST endpoint that is only exposed over Spring MVC or Spring WebFlux;

[@Selector](#) - works like `@PathVariable` annotation by binding a named URI actuator endpoint path parameter into an argument;

[@ReadOperation](#) – used to indicate a method on an `@Endpoint` component class as read operation (HTTP GET accessible);

[@WriteOperation](#) – used to indicate a method on an `@Endpoint` component class as write operation (HTTP POST accessible);

[@DeleteOperation](#) – used to indicate a method on an `@Endpoint` component class as delete operation (HTTP DELETE accessible);

12. SPRING BOOT TESTING

SAMPLE PROJECTS

- [spring-testing](#)

REFERENCES

- <https://spring.io/guides/gs/testing-web/>
- <https://howtodoinjava.com/spring-boot2/testing/springboot-test-configuration/>
- <https://howtodoinjava.com/spring-boot2/testing/junit5-with-spring-boot2/>
- <https://newbedev.com/unit-test-or-integration-test-in-spring-boot>
- <https://docs.spring.io/spring-framework/docs/current/reference/html/testing.html>
- <https://reflectoring.io/spring-boot-test/>
- <https://rieckpil.de/spring-boot-unit-and-integration-testing-overview/>
- <https://rieckpil.de/spring-boot-test-slices-overview-and-usage/>
- <https://rieckpil.de/guide-to-springboottest-for-spring-boot-integration-tests/>
- <https://rieckpil.de/difference-between-mock-and-mockbean-spring-boot-applications/>
- <https://rieckpil.de/fix-no-qualifying-spring-bean-error-for-spring-boot-tests/>

KEY ANNOTATIONS

@ExtendWith - JUnit annotation that is used to register extensions for the annotated test class or test method. Prior to Spring 2.1 release, `@ExtendWith(SpringExtension.class)` was necessary to be specified at the test class-level in order to enable Spring support, but as of then is no longer needed because it is included as a meta annotation in the Spring Boot test annotations like `@DataJpaTest`, `@WebMvcTest`, and `@SpringBootTest`.

@SpringBootTest - used to bootstrap a complete application context for testing.

`@SpringBootTest` by default starts searching in the current package of the annotated test class and then searches upwards through the package structure, looking for a class annotated with `@SpringBootConfiguration` from which it reads the configuration to create an application context. This class is usually the main application class since the `@SpringBootApplication` annotation includes the `@SpringBootConfiguration` annotation. It then creates an application context very similar to the one that would be started in a production environment.

@ContextConfiguration - used to load component classes¹ in order to configure an `ApplicationContext` for integration test.

@WebMvcTest - used to set up an application context with just enough components and configurations required to test the **Web MVC Controller Layer**. It disables full auto-configuration and instead apply only configuration relevant to MVC tests.

@DataJpaTest - used to set up an application context specifically to test the **Persistence Layer**, by configuring entities, repositories. `@DataJpaTest` by default autoconfigures an in-memory/embedded database like H2.

@Commit - *test annotation* used to indicate that changes performed on database from a test method should be committed before it ends. When used at class-level, all test methods will commit changes to the database.

@Rollback - *test annotation* used to indicate that changes performed on database from a test method should be roll backed before it ends. When used at class-level, all test methods will behave in that way.

@TestConfiguration - used to define additional beans or override existing beans in the Spring application context in order to add specialized configuration for testing. Configuration classes annotated with @TestConfiguration are excluded from component scanning. Configuration classes with bean definition annotated with @TestConfiguration can be imported by @Import or declared as static inner classes. It is required to set the property `spring.main.allow-bean-definition-overriding=true` in order to enable bean overriding feature during testing.

@MockBean - part of the Spring Test Framework and used to create mocks for beans whenever running tests targeted to the *Spring Test context* (i.e. testing with @SpringBootTest, @WebMvcTest, @DataJpaTest and so on).

@Mock - part of the **Mockito Framework** and used to create a mock for a class or interface. It is a shorthand for the `Mockito.mock()`.

@InjectMocks - part of the **Mockito Framework** and used to create a mock for the marked field and injects all dependencies annotated with @Mock into it.

NOTES

@SpringBootTest vs @ContextConfiguration

- Even though both can be used to specify how to load an application context in integration test, @ContextConfiguration doesn't take full advantage of Spring Boot features, like logging or additional property loading.

@MockBean vs @Mock

- For any test that doesn't need any dependencies from the Spring Boot container (application context), @Mock should be used as it is fast and favours the isolation of the tested component;
- If a test relies on the Spring Boot container (application context) and it's also needed to add or mock one of the container beans, @MockBean should be used;
- As a rule of thumb, @Mock should be used when testing services components where business logic are implemented, and @MockBean should be used when doing sliced context testing like on the controller (@WebMvcTest) or repository (@DataJpaTest) layer,

or when running backed Spring context tests (@SpringBootTest).

13. BUILD & TEST

BUILD ALL PROJECTS

```
$ ./mvnw package
```

BUILD ALL PROJECTS SKIPPING TEST

```
$ ./mvnw package -DskipTests
```

BUILD INDIVIDUAL PROJECT

```
$ ./mvnw package -pl spring-crud
```

BUILD INDIVIDUAL PROJECT SKIPPING TEST

```
$ ./mvnw package -DskipTests -pl spring-crud
```

RUN INDIVIDUAL PROJECT

- Option 1:

```
$ ./mvnw spring-boot:run -pl spring-crud
```

- Option 2:

```
$ java -jar spring-crud/target/spring-crud-0.0.1.jar
```

RUN ALL PROJECTS TEST

```
$ ./mvnw test
```

RUN INDIVIDUAL PROJECT TEST

```
$ ./mvnw test -pl spring-crud
```

RUN SINGLE TEST CLASS FROM INDIVIDUAL PROJECT

```
$ ./mvnw test -pl spring-crud -Dtest=io.davidarchanjo.controller.ToDoControllerTest
```

RUN SINGLE TEST METHOD FROM A TEST CLASS OF AN INDIVIDUAL PROJECT

```
$ ./mvnw test -pl spring-crud -Dtest=io.davidarchanjo.controller.ToDoControllerTest#shou
```

¹ component classes - is any class annotated with `@Configuration` or `@Component` (including any of its stereotype like `@Service`, `@Repository` etc) as well as any JSR-330 compliant class that is annotated with `javax.inject` annotations.

Releases

No releases published

Packages

No packages published

Contributors 2



davidarchanjo David Archanjo



CodePawfect

Languages

● Java 99.4% ● HTML 0.6%